

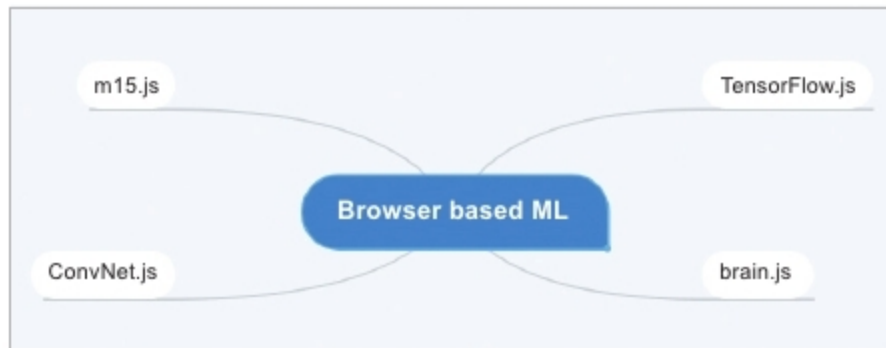


Figure 1: Web browser based machine learning libraries

follows:

```
<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@0.14.2/dist/tf.min.js"></script>
```

Another approach is to use NPM, as follows:

```
npm install @tensorflow/tfjs
```

If you want to convert an existing model, the *tensorflowjs_converter* can be used. The following code is used to convert the *SavedModel*:

```
tensorflowjs_converter \
--input_format=tf_saved_model \
--output_node_names='MobilenetV1/Predictions/Reshape_1' \
--saved_model_tags=serve \
/mobilenet/saved_model \
/mobilenet/web_model
```

In the case of a *FrozenModel*, the following command can be used:

```
tensorflowjs_converter \
--input_format=tf_frozen_model \
--output_node_names='MobilenetV1/Predictions/Reshape_1' \
/mobilenet/frozen_model.pb \
/mobilenet/web_model
```

To run this conversion, you require a Python environment. The converter can be installed with Pip:

```
pip install tensorflowjs
```

The conversion script generates three files:

- The dataflow graph (*web_model.pb*)
- The weight manifest file (*weights_manifest.json*)
- A collection of binary weight files (*group1-shard*of**)

The loading and running of the model in the browser can be done using the following code snippet:

```
import * as tf from '@tensorflow/tfjs';
import {loadFrozenModel} from '@tensorflow/tfjs-converter';
const MODEL_URL = 'https://.../mobilenet/web_model.pb';
const WEIGHTS_URL = 'https://.../mobilenet/weights_manifest.
```

```
json'; const model = await loadFrozenModel(MODEL_URL,
WEIGHTS_URL);
const cat = document.getElementById('cat');
model.execute({input: tf.fromPixels(cat)});
```

(Source: <https://js.tensorflow.org/tutorials/import-saved-model.html>)

The complete source code and execution procedure can be gathered from <https://github.com/tensorflow/tfjs-converter/tree/master/demo/mobilenet>.

A detailed tutorial on TensorFlow.js is available at <https://js.tensorflow.org/>.

ml5.js

An understanding of ML concepts is necessary to use libraries such as TensorFlow.js effectively. If you are not interested in the internals of how things work but you only want ML applications, then give ml5.js a try. The official documentation claims that this library is made to enable a broader audience get involved in machine learning. ml5.js is built on top of TensorFlow.js and is hugely inspired by 'processing' and p5.js.

Setting up your environment to use ml5.js is very simple. Just use a `<script>` tag to link the external JavaScript file.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Machine Learning with ml5.js</title>
    <script src="https://unpkg.com/ml5@0.1.3/dist/ml5.min.js">
    </script>
  </head>
</html>
```

A simple image classification example is shown below:

```
<body>
<h1>Image classification using MobileNet</h1>
<p>The MobileNet model labeled this as
<span id="result">...</span> with a confidence of
<span id="probability">...</span></p>


<script>
// The image we want to classify
const image = document.getElementById('image');
// The result tag in the HTML
const result = document.getElementById('result');
// The probability tag in the HTML
const probability = document.getElementById('probability');

// Initialize the Image Classifier method with MobileNet
```